

AWS CloudFront

How CloudFront Redirects & Routes Traffic

1. What is Amazon CloudFront?

Amazon CloudFront is a Content Delivery Network (CDN) that sits in front of your application. It distributes content from edge locations worldwide, reducing latency for end users. Beyond caching, CloudFront can intelligently route requests to different backends based on the URL path — serving static files from S3 and forwarding API calls to an ALB or API Gateway, all under a single domain.

2. Key Concepts

Origins

An **origin** is the backend server CloudFront fetches content from. You can define multiple origins in one distribution:

- **S3 Bucket** — for static assets (HTML, CSS, JS, images)
- **ALB / EC2** — for dynamic API requests
- **API Gateway** — for serverless APIs
- **Custom HTTP origin** — any public HTTP/HTTPS endpoint

Behaviors (Path-Based Routing)

A **behavior** maps a URL path pattern to an origin. CloudFront evaluates behaviors top-to-bottom and routes the request to the first matching origin. One behavior is always the **default (*)** catch-all.

Path Pattern	Origin	Cache	Use Case
/api/*	ALB / API Gateway	No (TTL=0)	Dynamic API calls
/static/*	S3 Bucket	Yes (long TTL)	CSS, JS, images
*.html	S3 Bucket	Short TTL	HTML pages
/* (default)	S3 Bucket	Yes	Everything else

3. How CloudFront Routes a Request (Step by Step)

Step 1 — User Request: User browser sends a request to yourapp.com/api/users.

Step 2 — DNS Resolution: yourapp.com resolves to CloudFront's edge location (nearest to the user).

Step 3 — Behavior Matching: CloudFront checks the path /api/users against all defined behaviors, top-to-bottom. It matches /api/* first.

Step 4 — Cache Check: CloudFront checks if the response is cached. For API calls (TTL=0), it skips cache and forwards the request.

Step 5 — Origin Fetch: CloudFront forwards the request to the mapped origin (e.g., ALB → ECS). Response is returned.

Step 6 — Response to User: CloudFront returns the response to the user. Static content may be cached at the edge for future requests.

4. Full Stack Architecture Example

A common pattern for React/SPA apps hosted on AWS:

```
User
  ■■■ CloudFront (yourapp.com)
  ■■■ /api/* ■■■ ALB ■■■ ECS (Node/Python API)
  ■■■ /* ■■■ S3 (React build: index.html, app.js, styles.css)
```

Benefits of this pattern:

- **Single domain** — no CORS issues between frontend and API
- **Edge caching** — static assets served in milliseconds globally
- **SSL termination** — CloudFront handles HTTPS for all routes
- **DDoS protection** — AWS Shield is built into CloudFront

5. Redirects vs. Forwarding

CloudFront supports two distinct mechanisms that are often confused:

Forwarding (Proxy)

CloudFront transparently proxies the request to the origin. The user's browser URL does not change. This is what behaviors do — /api/* is forwarded to the ALB silently.

HTTP Redirects

CloudFront can also issue HTTP 301/302 redirects. Common use cases include:

- **HTTP → HTTPS redirect:** Set viewer protocol policy to 'Redirect HTTP to HTTPS' on any behavior.
- **www → non-www:** Use a Lambda@Edge function triggered on viewer-request to issue a 301.
- **Custom redirects:** Lambda@Edge or CloudFront Functions can inspect the request and return redirect responses.

6. Cache Control for API Requests

API responses should typically not be cached. Configure the behavior for /api/* as follows:

```
Cache Policy: CachingDisabled (TTL = 0)
Origin Request Policy: AllViewerExceptHostHeader
Allowed Methods: GET, HEAD, OPTIONS, PUT, POST, PATCH, DELETE
Forward Headers: Authorization, Content-Type
```

7. Advanced: Lambda@Edge & CloudFront Functions

For custom routing logic beyond path matching, AWS provides two compute options at the edge:

Feature	CloudFront Functions	Lambda@Edge
Latency	Sub-millisecond	~1ms
Runtime	JS (ES5.1)	Node.js, Python
Max exec time	1ms	5–30 seconds
Use case	Simple rewrites, header edits, redirects	Complex logic, auth, A/B testing
Trigger points	Viewer request/response	All 4 event types